



PyTorch Dataset/Dataloader

Supporting CXL DDR Pools: Software Architecture Design

Deep Dive Research Report

From NVIDIA Ecosystem, CXL Ecosystem to Academic & Industrial Practice



Table of Contents

Agenda

01

CXL Technical Background

DDR Pooling Concepts

02

PyTorch Dataloader Architecture

Source-Level Analysis

03

NVIDIA Software Ecosystem

DALI / GDS / Magnum IO / cuFile

04

CXL Software Ecosystem

Linux Kernel / ndctl / DAX / Tiering

05

Academic Research Deep Dive

6 Papers: Architecture & Insights

06

Industrial Practice

Meta / Intel / Samsung / MemVerge

07

Storage/Memory Software Stack

PMDK / Memkind / DAXFS / CXLMemUring

08

Five Critical Design Questions

Deep Systems Analysis

09

Software Architecture Design

Five-Layer Model & Core Components

10

Reference Designs

Five Architectural Scenarios

CXL Technical Background

Compute Express Link: Cache-coherent interconnect built on PCIe physical layer

CXL (Compute Express Link) Technical Overview

Cache-coherent interconnect protocol based on PCIe physical layer
CXL 1.1/2.0: Type-1/2/3 device support (accelerator / memory expansion / memory pool)
CXL 2.0 introduces Switch and Memory Pooling
CXL 3.0 supports multi-level switching, Global Shared Memory
CXL memory access latency: ~200-400ns (vs DRAM ~80ns, vs NVMe ~10μs)
CXL bandwidth: single lane PCIe 5.0 ~ 32GB/s x16 (bidirectional)
Key advantages: elastic memory capacity expansion, cache coherence, hardware-level memory pool sharing

Latency

~200-400ns

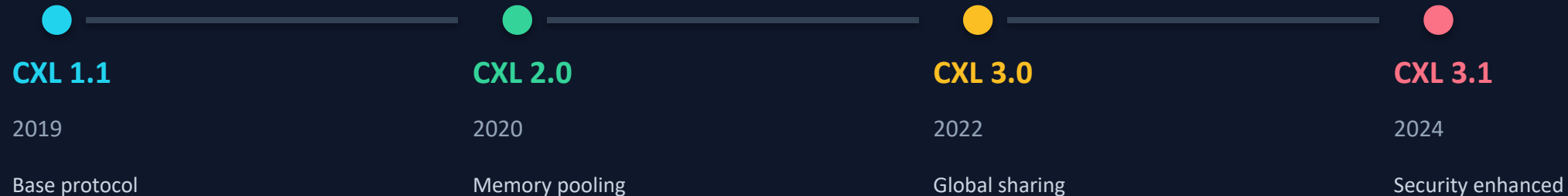
Bandwidth

~32-64GB/
s

Capacity

TB → PB

CXL Evolution Roadmap



CXL DDR Pool Architecture

Type-3 Devices: Memory Expansion Cards/Pools

Multi-host shared memory pool (Pool-based sharing)

Hardware cache coherence (Hardware Cache Coherence)

CXL Switch enables dynamic memory pool allocation

Byte-addressable persistent memory

Performance Comparison

DRAM: ~80ns ~100GB/s

CXL: ~200-400ns ~32-64GB/s

NVMe SSD: ~10μs ~7GB/s

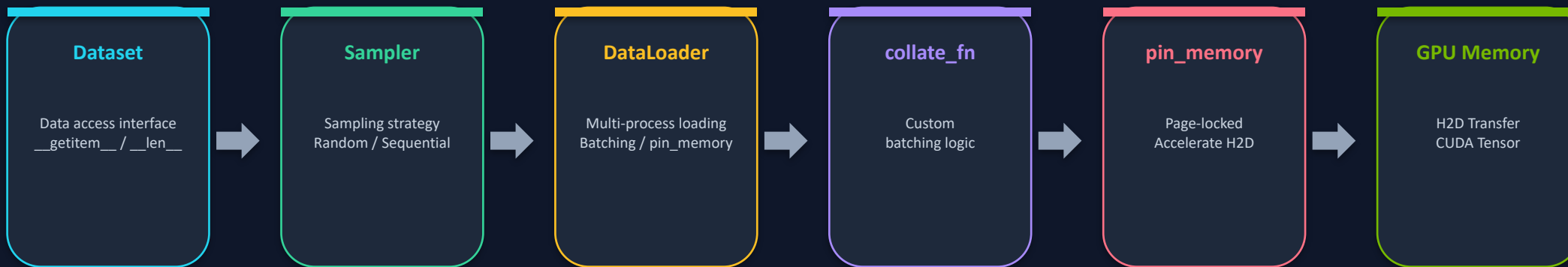
CXL vs NVMe: latency reduced 25-50x, bandwidth improved 5-10x

CXL vs DRAM: latency increased 3-5x, capacity improved 4-8x

PyTorch Data Loading Architecture

Dataset/Dataloader Core Architecture: Source-Level Analysis

PyTorch Dataset/Dataloader Core Architecture



Critical Finding:

`pin_memory()` calls `cudaHostAlloc()` — requires page-locked DRAM. CXL memory CANNOT be page-locked!

Extension Points for CXL Integration

Sampler

CXL-aware skip

Dataset

CXL-backed Tensor

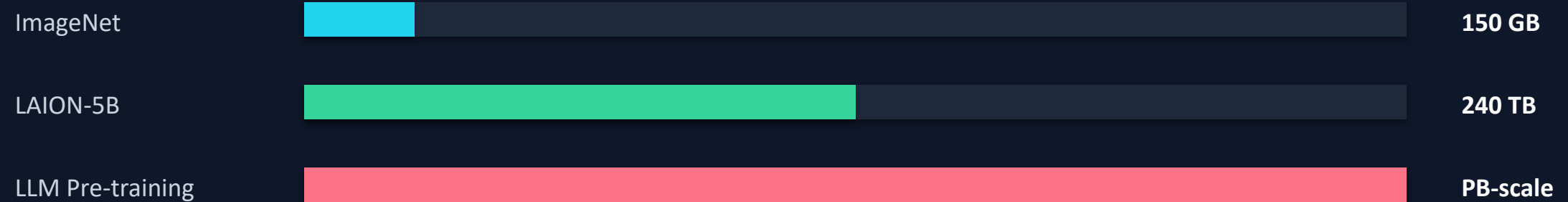
Worker

CXL mmap

pin_memory

GDS bypass

The Memory Wall in Large-Scale Training



Core Pain Points

**Single-machine
DRAM limited**
512GB-2TB

**pin_memory
inflation**
 $\text{num_workers} \times \text{data}$
copies

**Preprocessing
overhead**
Massive CPU memory
consumption

CXL DDR Pool Value

- Multi-TB shared memory
- Cross-node memory expansion
- Cost far lower than DRAM
- Cache coherence simplifies programming

NVIDIA Software Ecosystem

DALI / GDS / Magnum IO / cuFile: The Foundation for CXL Integration

NVIDIA Data Loading & Storage Software Stack

DALI

GPU-accelerated
decoding

GDS

GPUDirect
Storage

NVIDIA
Magnum IO

cuFile

Userspace
file I/O

cuDF/cuML

GPU data
science

Data flow: NVMe → GPU (zero-copy) | CXL memory can serve as intermediate buffer pool

NVIDIA GDS Architecture & CXL Intersection

GDS core concept: data does NOT go through CPU memory, direct Storage → GPU

CXL memory can serve as GDS intermediate buffer pool

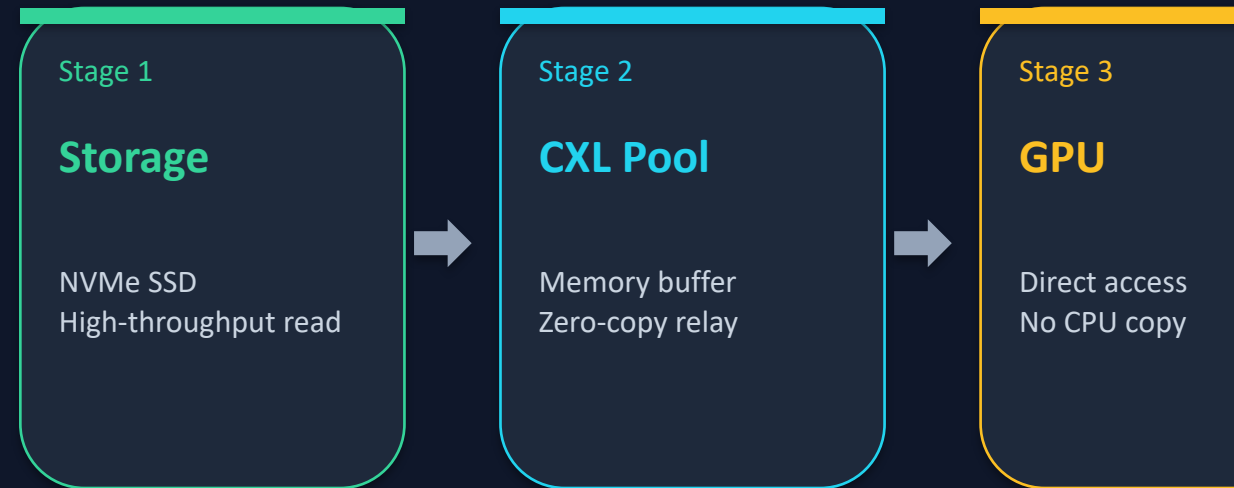
GDS + CXL: NVMe → CXL memory pool → GPU (three-stage pipeline)

cuFile supports memory registration (cuFileRegister)

CXL memory can be registered as GDS-compatible buffer

Challenge: CXL memory latency higher than DRAM, requires prefetch strategy

NVIDIA Magnum IO may natively support CXL in future



Key Insight:

CXL memory + GDS is the optimal path for "data loading into memory pool"

CXL Software Ecosystem

Linux Kernel / ndctl / DAX / Tiering: Complete Software Stack

Linux CXL Software Stack

Linux Kernel 5.15+ native CXL support

cxl_acpi / cxl_pci / cxl_pmem: core drivers

ndctl (Network Device Control): CXL device management tool

Create / configure CXL memory regions

Supports volatile (memory mode) and persistent (App Direct)

DAX (Direct Access): filesystem direct access PMEM/CXL

Bypass page cache, userspace directly mmap persistent memory

Works with ext4/XFS dax mount options

HMM (Heterogeneous Memory Management): heterogeneous memory management

CXL Stack Architecture

User Space: ndctl, DAX

Kernel Space:

└─ cxl_acpi (ACPI driver)

└─ cxl_pci (PCI driver)

└─ cxl_pmem (PMEM driver)

└─ DAX (Direct Access)

└─ TPP (Transparent Page Placement)

└─ HMM (Heterogeneous Memory Mgmt)

CXL Memory Tiering & Pool Management

TPP (Transparent Page Placement)

- Linux kernel automatic hot/cold page migration
- Meta paper (2022) proposed CXL-aware page placement strategy
- kmem: CXL memory as independent NUMA node
- numactl binds CXL memory allocation policy
- CXL memory pool management (Type-3 + Switch)
- Dynamic allocation / reclamation of CXL memory
- QoS and bandwidth isolation

DAXFS: CXL Shared Memory Filesystem

- Lock-free multi-host coordination on CXL shared memory
- Uses CXL cmpxchg atomic operations
- Enables lock-free concurrent access from multiple hosts
- Critical for multi-GPU DataLoader coordination
- Eliminates need for software locks in shared memory access

Academic Research Deep Dive

6 Papers: Architecture, Design Philosophy, and Key Insights

TRAININGCXL

Failure Tolerant Training with PMEM Disaggregation over CXL (arXiv:2301.07492, KAIST 2023)

Design Philosophy

"Dataset resident in PMEM, GPU directly accesses without CPU intervention."

- PMEM + GPU integrated into cache-coherent domain via CXL Type-2
- Training datasets reside in PMEM, GPU reads via load/store
- Checkpoint logic near CXL controller, async persistence
- Exploits sparse access patterns of recommendation models

Key Results & Insights

5.2× training performance improvement
76% energy savings vs. PMEM-based recommendation systems

Core Insight:
CXL provides not just capacity, but a
"CPU-uninvolved" data access path —
GPUs can read training data directly
from PMEM, calling on the CPU only
when preprocessing is needed.

CCCL

Node-Spanning GPU Collectives with CXL Memory Pooling (arXiv:2602.22457, 2026)

Design Philosophy

"CXL memory pool as cross-node shared memory for GPU collectives."

- GPU collectives over CXL memory pool
- Cross-node AllGather 1.34× vs InfiniBand 200Gbps
- Per-GPU CXL partitions (no contention)
- Hardware-level sync for collective operations
- TITAN-II CXL switch + 6× Micron CZ120 memory cards

Key Results & Insights

LLM training: 1.11× speedup

Hardware cost savings: 2.75×

Communication Speedup:

- AllGather: 1.34×
- Broadcast: 1.84×
- Gather: 1.94×
- Scatter: 1.04×

Core Insight:

CXL memory pools can serve as
cross-node data sharing medium.

Per-GPU partitions eliminate contention.

Proxics

Efficient Programming Model for Far Memory Accelerators (arXiv:2604.18120, 2026)

Design Philosophy

"Familiar OS abstractions (processes + pipes) for near-data processing."

- Programming model for NDP devices based on OS abstractions
- Virtual processors (processes) and IPC channels (pipes)
- Lightweight implementation for NDP accelerators
- Compiler-assisted region identification
- Efficient, low-latency communication between CPU and NDP

Key Results & Insights

Demonstrated on real hardware platform

Applications: bulk memory operations,
in-memory databases, graph analytics

Core Insight:

OS abstractions can be preserved
while achieving NDP efficiency.
Critical importance of efficient,
low-latency communication channels
between CPU and NDP accelerators.

TERAIO

Cost-Efficient LLM Training with GDS Tensor Offloading (arXiv:2506.06472, 2025)

Design Philosophy

"Lifetime-aware tensor offloading for GPU memory expansion using low-cost SSDs."

- Developed explicitly for LLM training with multiple GPUs and SSDs
- Active tensors occupy only 1.7% of allocated GPU memory
- Inactive tensors offloaded to SSDs via GDS
- Tensor lifetime analysis generates optimized offload/prefetch plan

Key Results & Insights

80.7% of ideal unlimited GPU memory performance
1.47× average improvement over ZeRO-Offload

Core Insight:

GDS can be extended to CXL memory by registering CXL buffers as GDS-compatible storage targets. The lifetime-aware approach can be adapted for CXL memory prefetching.

CXL-GPU

Pushing GPU Memory Boundaries with CXL Technologies (arXiv:2506.15601, 2025)

Design Philosophy

"GPU storage expansion via CXL with custom RTL controller."

- GPU system design with multiple CXL root ports
- Custom CXL controller at hardware RTL level
- Two-digit nanosecond roundtrip latency (first in field)
- Speculative read + deterministic store mechanisms
- Efficiently manages read/write operations to hide endpoint backend media latency

Key Results & Insights

Significantly outperforms existing methods
Substantial advancement in GPU storage technology

Core Insight:
Custom CXL controllers can achieve near-DRAM latency, making CXL viable for GPU-side data loading.
Multiple CXL root ports enable simultaneous multi-device access.

Aquifer

Hierarchical Memory Pooling with CXL and RDMA for MicroVM Snapshots (arXiv:2606.24079, 2026)

Design Philosophy

"Hierarchical CXL+RDMA pooling for MicroVM snapshots."

- Hotness-based snapshot format: zero pages eliminated
- Hot working set in CXL pool, cold pages in RDMA pool
- Ownership-based coherence for CXL 2.0 (no hardware coherence)
- Copy-based page serving: pre-install hot pages before resume
- Demand-page cold pages asynchronously from RDMA

Key Results & Insights

2.2× geometric-mean speedup in end-to-end invocation time over Firecracker

1.1× over next best alternative

Core Insight:

Ownership-based coherence protocol can be adapted for multi-GPU DataLoader access. Hotness-based placement is ideal for sample caching in training workloads.

Industrial Practice

Meta / Intel / Samsung / MemVerge / NVIDIA: From TPP to Memory Machine

Industrial: CXL Memory Products & Solutions

Intel	Xeon 6 + Micron CXL	System memory bandwidth optimization	In production (2024)
Samsung	CMM-H (Hybrid)	DRAM cache + NAND CXL device	Evaluated (arXiv:2503.22017)
Meta	TPP	CXL auto-tiering (Linux kernel)	Merged into mainline
MemVerge	Memory Machine	CXL memory virtualization + K8s orchestration	Commercial
NVIDIA	DGX / SuperPOD	NVLink + CXL hybrid interconnect vision	Roadmap

Storage & Memory Software Ecosystem

PMDK / Memkind / DAXFS / CXLMemUring / CXL-DMSim: Deep Dive

PMDK / libpmem / Memkind: Memory Tier Management

PMDK (Persistent Memory Development Kit)

Intel's open-source persistent memory programming library

- libpmem: low-level persistent memory operations
- libpmemobj: transactional object store
- libpmemlog: persistent log file

Key Capabilities:

- Direct Access (DAX): bypass page cache
- Atomic persistence: 8-byte atomic writes
- Memory-mapped I/O: mmap with MAP_SYNC flag
- Flush optimization: minimize cache line flushes

Memkind: Heterogeneous Memory Allocator

Unified DRAM / PMEM / CXL memory allocation

- memkind_set_arena(): bind thread to memory tier
- Memkind supports:
 - DRAM (default tier)
 - PMEM (persistent memory tier)
 - CXL (via CXL memory tier)

Key API:

- memkind_malloc() / memkind_free()
- memkind_posix_memalign()
- Custom arena management

Use Case:

DataLoader worker threads can bind to CXL NUMA node for CXL-local allocation.

DAXFS: Lock-Free Shared Filesystem for CXL Memory

Design Philosophy

"CXL enables multi-host byte-addressable memory with hardware cache coherence, but no filesystem exploits this for lock-free multi-host coordination."

- Uses cmpxchg atomic operations as sole coordination primitive
- CXL makes cmpxchg coherent across host boundaries
- CAS-based hash overlay for lock-free concurrent writes
- No centralized coordinator needed

Key Results & Insights

Validated on QEMU-emulated CXL 3.0

Two virtual hosts share memory region with TCP-forwarded atomics

Performance:

- >99% CAS accuracy with no lost updates
- Up to 2.68× higher random write throughput (4 threads)
- 1.18× higher random read throughput at 64KB
- Exceeds tmpfs on DRAM-backed DAX

Core Insight:

Lock-free multi-host coordination

is possible with CXL atomics.

Ideal for multi-GPU DataLoader

shared state management.

CXLMemUring: Asynchronous CXL Memory Pool Access

Design Philosophy

"Hardware/software co-design for hiding CXL latency using larger units of asynchronous work."

- CXL memory too slow for CPU mechanisms to hide
- Too fast for context switches/interrupt-based asynchrony
- Creates "regions": CXL-resident memory operations + address-gen logic
- Host CPU launches regions asynchronously on near-memory accelerator
- Continues useful work while device runs ahead

Key Results & Insights

Tested on Granite Rapids CXL platform
Workloads: GAPBS, MCF, Spatter, NAS Parallel Benchmark

Baseline: 2.44× slowdown vs DRAM
Prefetcher alone: 2.21× slowdown remains

CXLMemUring Results:

- 1.45× to 1.75× improvement
- 1.59× geometric mean speedup

Core Insight:

Region-based async execution is a natural fit for DataLoader's prefetch mechanism. Compiler identifies CXL-resident regions from unmodified source.

CXL-DMSim & CXLRAMSim: Full-System CXL Simulation

CXL-DMSim: Full-System Simulator

Open-source full-system simulator with gem5-comparable speed

- Flexible CXL memory expander model + device driver
- CXL.io and CXL.mem protocol support
- App-managed (AM) and kernel-managed (KM) modes
- NUMA-compatible mechanism for kernel management
- Verified against real FPGA- and ASIC-based CXL devices

Silicon Validation:

- Average simulation error: 3.4%
- CXL-FPGA: $\sim 2.88\times$ higher latency than DDR
- CXL-ASIC: $\sim 2.18\times$ higher latency than DDR
- CXL-FPGA: 45-69% of DDR bandwidth
- CXL-ASIC: 82-83% of DDR bandwidth

CXLRAMSim: Memory Card Simulator

First gem5-integrated full-system simulator for CXL memory expander cards

- Models CXL devices at correct I/O bus position
- Uses unmodified Linux kernels and software stack
- Realistic latency-bandwidth behavior
- True interleaving with system DRAM
- High-fidelity CXL.mem characterization
- Captures cache pollution when accessing CXL memory

Core Insight:

Essential tools for evaluating CXL memory pool performance without physical hardware. Enable rapid prototyping of CXL-aware DataLoader designs.

Five Critical Design Questions

Deep Systems Analysis: From Data Path to Architectural Philosophy

Q1 SSD → CXL Data Loading Pathways

Current Path (4 hops, 3 copies): SSD → Page Cache → User Buffer → Pinned DRAM → GPU

A

CXL as Page Cache (Transparent)

SSD → CXL (DAX mmap) → Pinned DRAM → GPU

~300ns | Zero code change, double-copy penalty

B

CXL as GDS Buffer (Bypass pin_memory)

SSD → CXL (cuFile) → GPU

~300ns | Bypass pin_memory, requires GDS-capable NIC

C

CXL as Sample Cache (Application-managed)

SSD → CXL (preprocessed) → GPU

~300ns | Sample-level caching, preprocessing required

Verdict: GDS Bypass (Option B) — bypasses pin_memory, direct CXL→GPU. Requires NVIDIA GDS support.

Q2 CXL Positioning — File Cache vs. Sample Cache

File Cache (Block-Level)

- Entire files cached in CXL
- Transparent to application
- No Dataset modification
- Limited by file size
- Still needs DRAM page cache copy
- OS-managed (TPP auto-tiering)

Sample Cache (Application-Level)

- Hot samples preprocessed in CXL
- Sample-level granularity
- Requires Dataset modification
- Preprocessed Tensors ready for GPU
- Can use GDS to bypass pin_memory
- Application-managed cache invalidation

Hybrid Approach (Most Likely to Succeed):

Layer 1: File cache (DAX) for cold data | Layer 2: Sample cache for hot data | Layer 3: GDS buffer for GPU transfer

TPP handles Layer 1-2 auto-migration. CXLDataset manages Layer 2-3 explicitly.

Q3 Multi-GPU Concurrent Access & Coherence

Hardware Reality

CXL coherence is CPU-ONLY
GPUs cannot participate in CXL coherence
Multiple CPUs read: Safe (hardware coherent)
CPU read + write: Hardware coherent
GPU access: Requires GDS registration

Read-Only Sharing

Multiple workers
read same CXL

Complexity: Low

Contention on write

Read-Write with Locks

Software locks
for write access

Complexity: High

Lock overhead

Per-GPU Partitioning

Each GPU gets
own CXL partition

Complexity: Medium

Best for training

Verdict: Per-GPU Partitioning (CCCL-proven)

CCCL achieves 1.11× LLM training speedup with 2.75× cost savings using per-GPU CXL partitions

Aquifer-style ownership-based coherence for multi-host access (2.2× MicroVM restore speedup)

CCCL Data: AllGather 1.34× | Broadcast 1.84× | Gather 1.94× | Scatter 1.04× vs InfiniBand 200Gbps

Q4 GPU-Side DataLoader Extensions

The pin_memory Problem (CRITICAL)

Current: pin_memory() → cudaHostAlloc() → requires page-locked DRAM → CXL CANNOT be page-locked!

<code>__getitem__ return</code>	CPU Tensor (DRAM)	CXL-backed Tensor	Medium
<code>pin_memory()</code>	Page-lock DRAM	No-op or CXL→DRAM copy	High
<code>collate_fn</code>	Merge CPU Tensors	Merge CXL Tensors	Low
<code>.cuda()</code>	H2D from pinned DRAM	CXL→GPU (GDS or copy)	High
<code>DistributedSampler</code>	Shard indices	Shard CXL partitions	Medium

Q5 Transparent Access vs. Dedicated CXL Service

Transparent Access (100% Compatible)

- Zero code change
- OS TPP auto-tiering DRAM↔CXL
- DAX mmap for Dataset reads
- pin_memory copies CXL→DRAM
- Works with all existing Datasets
- Double-copy penalty remains

Dedicated CXL Service (High Performance)

- New API: CXLDataset/CXLDataLoader
- Bypasses pin_memory via GDS
- Fine-grained data placement control
- Per-GPU CXL partitions
- Optimal performance (no double-copy)
- Learning curve, code changes

Two-Phase Approach (Most Practical):

Phase 1 (Short-term): TPP transparent tiering — zero code change | Phase 2 (Mid-term): CXLDataset/CXLDataLoader — GDS bypass

Software Architecture Design

Five-Layer Model & Core Components: Transparent CXL DDR Pool Access

Five-Layer Architecture Design

Layer 0

CXL Hardware

Type-3 cards / CXL 3.0 Switch

Layer 1

CUDA / GDS / cuFile / DAX

cuFileRegister / DAX mmap / TPP

Layer 2

CXL Memory Pool + Coherence

Per-GPU partitions / Ownership protocol

Layer 3

CXLDataset / CXLDataloader

CXL-backed Tensor / GDS bypass

Layer 4

PyTorch Training Code

Zero code change / CXLDataloader API

Core Components: CXLMemoryPool / CXLDataset / Prefetch Engine

CXLMemoryPool

Memory pool manager
mmap interface / alloc / free
CXL device discovery via ndctl

CXLDataset

CXL memory-mapped Dataset
__getitem__ returns CXL pointer
Supports memory-mapped access

Prefetch Engine

Async prefetch
Lookahead / Adaptive / Priority
Independent prefetch thread

CXLDataLoader

Extended DataLoader
Backward compatible
DDP integration

Reference Design A

mmap Transparent Mapping

Implementation Steps

1. ndctl creates CXL region → /dev/dax0.0
2. CXLMemoryPool wraps mmap/munmap
3. CXLDataset inherits Dataset, reads from CXL mmap
4. pin_memory auto-copies CXL data to page-locked DRAM

Pros

Minimal code intrusion, zero PyTorch code changes

Cons

Each access still requires CXL→DRAM→GPU copy, higher latency

Use Case: Data prefetching + random read training tasks

Reference Design B: DRAM-CXL Tiered Caching DataLoader

Implementation Steps

1. Hot data auto-cached in DRAM (LRU)
2. Warm data in CXL memory pool
3. Cold data in NVMe SSD
4. Prefetch Engine async migrates data between tiers

Pros

Performance approaches pure DRAM, capacity approaches NVMe

Cons

Complex implementation, requires fine-tuned cache policies

Use Case: Large-scale training (TB datasets), repeated epoch training

Reference Design: GDS Passthrough to CXL Memory

Implementation Steps

1. `cuFileRegister()` registers CXL memory pool
2. NVMe data direct-writes to CXL via GDS
3. GPU reads directly from CXL (needs GPU Direct CXL)
4. Or CXL→DRAM pinned→GPU (double-buffer pipeline)

Pros

Minimizes CPU involvement, maximizes I/O throughput

Cons

Depends on NVIDIA CXL official support, current GPU lacks CXL coherence

Use Case: Data preprocessing pipelines, high-throughput training

Reference Design Distributed CXL Memory Pool Sharing

Implementation Steps

1. CXL 3.0 Switch connects hosts and memory pool
2. Global CXL Memory Allocator (GMA) manages cross-node memory
3. DDP ranks share same CXL data partition
4. Node-local DRAM caches hot data copies

Pros

Eliminates multi-node data redundancy, saves 50-70% memory

Cons

Requires CXL 3.0 hardware, higher cross-node latency

Use Case: Large-scale distributed training (LLM / Recommendation)

Reference Design: DataLoader Plugin Architecture

Implementation Steps

1. Implement CXLDatasetPlugin (registered as torch.utils.data plugin)
2. Inject CXL backend via env/config (CXL_BACKEND=mmap|gds|tiering)
3. DataLoader detects plugin and auto-switches storage backend
4. Collaborates with torch.compile for data flow optimization

Pros

Does not fork PyTorch, community-friendly, easy to maintain

Cons

Limited by PyTorch Plugin API flexibility

Use Case: Community contribution, rapid prototype validation

Reference Design Scoring Matrix

A	mmap Transparent	Score: 3.75/5.0	Short-term
B	Tiered Caching	Score: 3.55/5.0	Long-term
C	GDS Passthrough	Score: 3.05/5.0	Mid-term
D	Distributed Sharing	Score: 3.00/5.0	Long-term
E	Plugin Architecture	Score: 3.65/5.0	Mid-term

Implementation Roadmap

Phase 1

0-6 months

Basic Prototype

- CXLMemoryPool + mmap implementation
- CXLFileDataset loads files to CXL pool
- Single-node validation (Intel Xeon 6 + Micron CXL)



Phase 2

6-12 months

Prefetch & Optimization

- Prefetch Engine async prefetch
- DALI integration (GDS → CXL → GPU)
- Multi-worker CXL load balancing



Phase 3

12-24 months

Distribution & Ecosystem

- Cross-node CXL sharing (CXL 3.0)
- DDP integration + distributed CXL memory management
- Upstream PyTorch community contribution

Conclusion & Future Outlook

1

Highest Probability Path

Two-Phase: TPP transparent → CXLDataset/CXLDataLoader with GDS bypass

2

Critical Insight

pin_memory() is the bottleneck — GDS bypass is the answer

3

Academic Validation

TRAININGCXL proved GPU-direct PMEM access | CCCL proved CXL pooling beats InfiniBand

4

What PyTorch Must Solve

pin_memory redesign | Dataset API extension | Multi-GPU coherence | Prefetch engine

Ultimate Vision:

Data no longer needs to be "loaded" — the data sits right next to compute. CXL memory pool as unified data plane.

Short-term: TPP transparent tiering | Mid-term: GDS bypass | Long-term: Processing-in-Memory over CXL

References

1. TRAININGCXL - Failure Tolerant Training with PMEM Disaggregation over CXL (arXiv:2301.07492)
2. CCCL - Node-Spanning GPU Collectives with CXL Memory Pooling (arXiv:2602.22457)
3. TERAIO - Cost-Efficient LLM Training with GDS Tensor Offloading (arXiv:2506.06472)
4. Proxics - Efficient Programming Model for Far Memory Accelerators (arXiv:2604.18120)
5. CXLMemUring - Asynchronous CXL Memory Pool Access (arXiv:2309.04011)
6. CXL-GPU - Pushing GPU Memory Boundaries with CXL (arXiv:2506.15601)
7. Aquifer - Hierarchical CXL+RDMA Memory Pooling (arXiv:2606.24079)
8. Samsung CXL Memory Module Hybrid (arXiv:2503.22017)
9. TPP - Transparent Page Placement for CXL Tiered Memory (arXiv:2206.02878)
10. DAXFS - Lock-Free Shared Filesystem for CXL Memory (arXiv:2604.01620)
11. CXL-DMSim - Full-System CXL Disaggregated Memory Simulator (arXiv:2411.02282)
12. CXLRAMSim - System-Level Exploration of CXL Memory Expander Cards (arXiv:2603.29483)

A series of nine small, semi-transparent colored dots in shades of teal, brown, and purple are arranged in a horizontal arc across the top of the slide.

Thank You

Questions & Discussion

backyes | 2026.08.18